Rails at network speed

Building an interactive network inventory with Turbo

Turbo Drive · Turbo Frames · Turbo Streams · Stimulus

Netdeck — Rails 8.1

What are we building?

A single-model network management application with:

- A searchable, filterable, sortable host inventory
- Pagination that preserves browser history
- Normal RESTful create, show, edit, update, and destroy flows
- In-place record editing
- An on-demand status modal
- Toast notifications from redirects and stream updates

> The goal is SPA-like interaction while keeping Rails responsible for HTML and application state.

The Hotwire mental model

```
Turbo Drive
  └── accelerates full-page navigation

Turbo Frames
  └── define independently navigable page regions

Turbo Streams
  └── perform precise DOM mutations from the server

Stimulus
  └── adds small browser-only behaviors
```

HTML remains the application protocol.

There is no client-side copy of the domain model, router, or validation rules.

1. Turbo Drive: the baseline

Ordinary Rails links and forms already participate in Turbo Drive:

```
<%= link_to "+ Add host", new_network_host_path,
      class: "button" %>

<%= form_with model: network_host,
      class: "record-form" do |form| %>
  ...
<% end %>
```

The controller remains conventional Rails:

```
if @network_host.save
  redirect_to @network_host,
    notice: "Network host was created."
end
```

Benefit: fast navigation without replacing REST, redirects, validations, or server-rendered views.

2. A Turbo Frame defines a contract

```
<%= turbo_frame_tag "network_hosts_table",
      data: { turbo_action: "advance" },
      aria: { live: "polite" } do %>
  <div class="table-card">
    ...table rows...
  </div>

  <%= render "shared/tables/pagination",
        page: @table_page,
        frame: "network_hosts_table" %>
<% end %>
```

A request targeting `network_hosts_table` must return:

```
<turbo-frame id="network_hosts_table">...</turbo-frame>
```

Turbo extracts that frame and replaces only the matching region.

3. Controls can target a frame

The reusable filter form lives **outside** the results frame:

```
<%= form_with url: url, method: :get,
      aria: { controls: frame },
      data: {
        controller: "table-filters",
        turbo_frame: frame,
        turbo_action: "advance"
      } do |form| %>
  <%= form.search_field :query,
        data: { action: "input->table-filters#submit" } %>
<% end %>
```

- The input remains focused while results change
- Only the table enters a busy/loading state
- The form still works as an ordinary GET without JavaScript

A useful Frame design lesson

Frame boundaries are UX boundaries

Originally, the search input was inside the frame it updated:

```
[ network_hosts_table frame                    ]
[  search input                                 ]
[  results                                      ]
```

Every response replaced the focused input.

The corrected structure:

```
[ stable search controls ] ——targets——▶ [ results frame ]
```

Choose a frame boundary around what should change—not around everything involved in the interaction.

4. URLs remain the source of table state

The server composes a normal Active Record relation:

```
def table_records
  scope = table_model.filtered_by(table_filter_parameters)
  scope = scope.sorted_by(
    sort: params[:sort], direction: params[:direction]
  )

  @table_page = TablePage.new(
    scope, page: params[:page], page_size: params[:per_page]
  )
  @table_page.records
end
```

Filtering, sorting, and pagination are represented in query parameters.

Benefits: bookmarkable URLs, refresh safety, back/forward navigation, and no hidden client state.

Browser history without full-page loads

```
data: {
  turbo_frame: "network_hosts_table",
  turbo_action: "advance"
}
```

`turbo_action="advance"` promotes frame navigation into browser history.

1. User filters by `status=active`
2. Turbo replaces only the table
3. Address bar reflects the query
4. Back restores the previous table state
5. A copied URL renders the same state on the server

> Fast partial navigation does not have to mean disposable navigation state.

5. Reuse starts at the model boundary

Each model explicitly declares safe capabilities:

```
class NetworkHost < ApplicationRecord
  include Addressable, Filterable, Sortable

  filters_by :status, :location, :device_type
  searches_by :hostname, :design_name, :ip_address,
    :mac_address, :vendor, :hardware_model
  sorts_by :hostname, :design_name, :ip_address,
    :location, :device_type, :status, :updated_at,
    default: :hostname
end
```

Future models opt in using the same small DSL.

No controller trusts arbitrary column names from the request.

Safe filtering and search

```ruby
def filtered_by(filters = {})
  filters = filters.to_h.stringify_keys
  relation = all

  if filters["query"].present? && searchable_attributes.any?
    relation = matching(relation, filters["query"])
  end

  filterable_attributes.each do |attribute|
    if filters[attribute].present?
      relation = relation.where(attribute => filters[attribute])
    end
  end

  relation
end
```

Search uses `sanitize_sql_like`; filters use Active Record hashes.

Benefit: reusable behavior with an explicit security whitelist.

The default sort is behavior and presentation

```ruby
def sorted_by(sort: nil, direction: nil)
  attribute = sort_attribute_for(sort)
  direction = sort_direction_for(direction)

  order(attribute => direction).order(id: direction)
end
```

The same effective state drives the header helper:

```
Hostname ↑   Design name ↕️   IP address ↕️
```

- Stable secondary ordering by `id`
- Invalid columns fall back to the configured default
- `aria-sort` exposes state to assistive technology

6. Frames can load UI on demand

Every row can target an initially empty modal frame:

```
<%= link_to "Change status",
      edit_network_host_status_path(network_host),
      data: { turbo_frame: "modal" } %>

<%= turbo_frame_tag "modal" %>
```

The response supplies the matching frame:

```
<%= turbo_frame_tag "modal" do %>
  <%= render layout: "shared/modal",
        locals: { title: "Change status" } do %>
    ...form...
  <% end %>
<% end %>
```

Benefit: modal HTML and data are fetched only when requested.

Keep modal operations RESTful

```ruby
# config/routes.rb
resources :network_hosts do
  resource :status,
    only: %i[ edit update ],
    module: :network_hosts
end
```

```
GET   /network_hosts/:network_host_id/status/edit
PATCH /network_hosts/:network_host_id/status
```

Why a nested `status` resource?

- Avoids a custom `change_status` action
- Uses standard `edit` and `update` semantics
- Gives focused authorization and strong parameters a natural home
- Works with or without Turbo

7. One response, several precise changes

A successful modal update returns three stream operations:

```
<%= turbo_stream.replace dom_id(@network_host, :status) do %>
  <%= render "network_hosts/status",
        network_host: @network_host %>
<% end %>

<%= turbo_stream.replace "modal" do %>
  <%= turbo_frame_tag "modal" %>
<% end %>

<%= turbo_stream.prepend "toasts" do %>
  <%= render "shared/toast", message: flash[:notice] %>
<% end %>
```

One request updates the row, closes the modal, and confirms success.

Turbo Stream actions

Turbo provides server-rendered DOM operations:

Action	Typical use
`append` / `prepend`	Add rows, events, notifications
`replace`	Swap a component or frame
`update`	Replace a target's children
`remove`	Delete a row or message
`before` / `after`	Insert adjacent content
`refresh`	Trigger a page refresh/morph

Targets use normal DOM IDs—Rails helpers such as `dom_id` keep them predictable.

> Streams describe **what changed**, not how to reconstruct application state in JavaScript.

In-place editing uses the same pattern

```
<%= turbo_stream.replace @network_host do %>
  <%= turbo_frame_tag @network_host do %>
    <%= render @network_host %>
  <% end %>
<% end %>

<%= turbo_stream.prepend "toasts" do %>
  <%= render "shared/toast",
        message: flash[:notice], type: :notice %>
<% end %>
```

Notice that the replacement restores the frame itself.

Frame identity is preserved, so the record can be edited in place again.

8. Flash messages become Turbo toasts

The layout provides one stable stream target:

```
<section id="toasts" class="toasts"
  aria-live="polite" aria-atomic="false">
  <% flash.each do |type, message| %>
    <%= render "shared/toast",
          message: message, type: type %>
  <% end %>
</section>
```

Two delivery paths, one component:

- **Turbo Drive redirect:** Rails flash renders in the next layout
- **Frame/Stream update:** response prepends the toast immediately

This keeps notification rendering server-owned and consistent.

9. Stimulus handles browser-only behavior

```
// table_filters_controller.js
submit() {
  clearTimeout(this.timeout)
  this.timeout = setTimeout(
    () => this.element.requestSubmit(), 250
  )
}
```

```
// modal_controller.js
connect() {
  if (!this.element.open) this.element.showModal()
}
```

```
// toast_controller.js
resume() {
  this.timeout = setTimeout(
    () => this.dismiss(), this.durationValue
  )
}
```

No fetching, JSON parsing, routing, validation, or HTML templates in JavaScript.

Progressive enhancement by construction

With Turbo and Stimulus	Without JavaScript
Table updates in its frame	GET form loads a full results page
Modal opens as a dialog	Status editor renders as a page
Stream changes one status cell	PATCH redirects to the host
Toast arrives immediately	Flash appears after redirect
Turbo Drive accelerates links	Browser follows ordinary links

The fallback is not a second application. It is the same Rails application.

Test the HTML contract

```
patch network_host_status_url(@network_host),
  params: { network_host: { status: "maintenance" } },
  headers: { "Accept" => "text/vnd.turbo-stream.html" }

assert_select "turbo-stream[action='replace']" \
  "[target='status_network_host_#{@network_host.id}']"
assert_select "turbo-stream[action='replace'][target='modal']"
assert_select "turbo-stream[action='prepend'][target='toasts']"
```

Tests verify observable HTTP and DOM behavior—not framework internals.

The application currently has **19 tests and 82 assertions**.

Why this approach is good

- **Less duplicated state** — the database and URL remain authoritative
- **Less JavaScript** — Rails renders the UI; Stimulus handles local behavior
- **Familiar architecture** — routes, controllers, models, partials, redirects
- **Secure by default** — strong params and explicit filter/sort whitelists
- **Accessible foundations** — links, forms, native dialog, live regions
- **Easier debugging** — inspect an HTTP response and rendered HTML
- **Incremental adoption** — Drive first, Frames where useful, Streams where precise

> Hotwire makes the server-rendered application more capable instead of replacing it.

Tradeoffs and design discipline

Turbo removes a lot of client complexity, but not the need for design:

- Frame responses must include the expected frame ID
- Frame boundaries must match what should actually be replaced
- Stream targets must exist and have stable DOM IDs
- Filtered views need thought when an update changes filter membership
- History behavior should be chosen deliberately (`advance` vs `replace`)
- Complex offline-first or highly client-stateful tools may still need more JavaScript

Turbo is strongest when interactions are fundamentally navigation and CRUD.

Suggested live demo

1. Navigate between Hosts, New, Show, and Back — **Turbo Drive**
2. Type in search while keeping focus — **targeted Frame navigation**
3. Sort and paginate, then use Back — **history promotion**
4. Open Change status — **lazy modal Frame**
5. Submit a status — **three Turbo Stream mutations**
6. Watch and dismiss the confirmation — **streamed toast + Stimulus**
7. Open a host and edit in place — **record-specific Frame**
8. Disable JavaScript and submit a form — **progressive enhancement**

Keep the HTML

Add speed and precision where users benefit

Turbo Drive for navigation

Turbo Frames for boundaries

Turbo Streams for change

Stimulus for browser behavior

`bin/rails server` · open `http://localhost:3000`